



WILLSIM

FICHE DE COURS · RESSOURCE PÉDAGOGIQUE GRATUITE

Commande de moteurs et régulation PID

Faire tourner un moteur à la vitesse voulue, quelle que soit la charge, est le geste de base de la mécatronique. Cette fiche construit pas à pas une boucle de vitesse fiable, du modèle du moteur à courant continu au régulateur PID numérique, puis montre ce que l'intelligence artificielle peut y ajouter.

NIVEAU

Débutant

DURÉE

6 h (3 séances de 2 h)

PRÉREQUIS

Bases d'électricité (tension, courant) et d'Arduino

OUTILS

Arduino ou ESP32, motoréducteur à codeur, pont en H (TB6612FNG ou L298N)

OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- relier tension, courant et vitesse d'un moteur à courant continu par un modèle du premier ordre ;
- commander le sens et la vitesse d'un moteur par MLI et pont en H, en sécurité ;
- compter les fronts d'un codeur en quadrature par interruptions et en déduire une vitesse ;
- programmer un PID discret (saturation, anti-emballement, dérivée filtrée), le régler et le valider par des critères chiffrés ;
- situer ce que l'IA apporte à la commande et ce qu'elle ne remplace pas.

Déroulé indicatif : séance 1, sections 1 et 2 (modèle, MLI, pont en H, sécurité) ; séance 2, sections 3 et 4 (codeur, boucle fermée, PID) ; séance 3, sections 5 et 6 (réglage, exercice guidé, apport de l'IA).

1. Modèle simplifié du moteur à courant continu

Un moteur à courant continu (MCC) à balais transforme un courant en couple et, en tournant, produit une tension qui s'oppose à celle qu'on lui applique. Son modèle électromécanique :

$u = Ri + L \frac{di}{dt} + e$	Loi des mailles : tension u , résistance R , inductance L , courant i .
$e = K_e \omega$	Force contre-électromotrice, proportionnelle à la vitesse angulaire ω (rad/s).
$\Gamma = K_t i$	Couple proportionnel au courant ; en SI, K_t (N·m/A) = K_e (V·s/rad).
$J \frac{d\omega}{dt} = \Gamma - f\omega - \Gamma_r$	Inertie J ramenée à l'arbre, frottement visqueux f , couple résistant Γ_r .

La constante de temps électrique L/R étant en général bien plus courte que la constante de temps mécanique, on néglige L . En éliminant i et e , à couple résistant nul, il reste un système du premier ordre :

$$\tau \frac{d\omega}{dt} + \omega = Ku \quad \text{avec} \quad K = \frac{K_t}{Rf + K_t K_e}, \quad \tau = \frac{RJ}{Rf + K_t K_e}$$

Soit $H(p) = K / (1 + \tau p)$: après un échelon de tension, la vitesse parcourt 63 % de sa variation finale en τ (quelques dizaines de millisecondes pour un petit motoréducteur) et 95 % en 3τ .

Courant de blocage. Au démarrage ou arbre bloqué, $e = 0$: seul R limite le courant, $I_{\text{bloc}} = U / R$, soit 3 A pour $R = 4 \Omega$ sous 12 V, souvent plus de dix fois le courant à vide. Dimensionnez pont en H, alimentation et câbles en conséquence.

2. MLI (PWM) et pont en H

2.1 La modulation de largeur d’impulsion

Un microcontrôleur ne délivre pas une tension variable de puissance : il commute. La MLI (PWM, *pulse width modulation*) applique la pleine tension U pendant une fraction α de chaque période ; bobinage et inertie lissent courant et vitesse, si bien que le moteur réagit, en première approximation, à la tension moyenne αU . Avec `analogWrite(broche, n)`, $\alpha = n/255$. Sur Arduino Uno, cette MLI tourne à environ 490 Hz (980 Hz sur les broches 5 et 6) et fait siffler le moteur ; avec un TB6612FNG, la ligne `TCCR1B = (TCCR1B & 0xF8) | 0x01;` dans `setup()` porte les broches 9 et 10 à 31,4 kHz, inaudible.

2.2 Le pont en H

Pour changer de sens, on inverse la tension aux bornes du moteur grâce aux quatre interrupteurs électroniques du pont en H (figure 1). Fermer les deux interrupteurs du bas freine le moteur ; fermer ceux d’un même bras (S1 et S2, ou S3 et S4) court-circuite l’alimentation. Un circuit intégré pilote donc les interrupteurs, interdit cette combinaison et gère les temps morts.

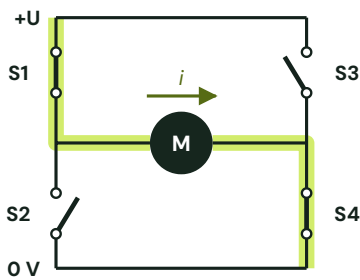


Figure 1 — S1 et S4 fermés : le courant suit le trajet surligné. S2 et S3 fermés : il traverse le moteur dans l’autre sens.

Commande d’une voie du TB6612FNG (STBY à l’état haut)

IN1	IN2	PWM	Effet
H	L	H	rotation dans un sens
L	H	H	rotation dans l’autre sens
IN1 ≠ IN2		L	frein (état bas de la MLI)
H	H	H ou L	frein (bornes en court-circuit)
L	L	H	roue libre (sorties coupées)

Ce freinage pendant l’état bas de la MLI rend la vitesse à peu près proportionnelle à α . Selon leurs fiches techniques, le TB6612FNG (MOSFET) admet 13,5 V et 1 A par voie en continu, avec peu de pertes ; le L298 (bipolaire, MLI sur ENA) monte à 46 V et 2 A par voie, mais perd environ 2 V à 1 A et doit être refroidi.

ATTENTION · SÉCURITÉ DU MONTAGE

- Alimentez le moteur par une source séparée (bloc protégé ou batterie avec fusible), jamais par le 5 V de l’Arduino ou l’USB ; reliez les masses.
- Câblez hors tension ; gardez à portée de main un interrupteur qui coupe la puissance : c’est l’arrêt d’urgence.
- Fixez le moteur, qui peut démarrer dès la mise sous tension ; éloignez doigts, cheveux et manches des pièces en rotation.
- Bloqué, le moteur appelle son courant de blocage et chauffe, le pont aussi : coupez au moindre bruit ou odeur anormale.
- Batterie au lithium : chargeur adapté, charge surveillée sur support ininflammable ; écarter toute batterie gonflée ou abîmée.

3. Codeur en quadrature et mesure de vitesse

3.1 Deux signaux décalés d'un quart de période

Un codeur incrémental monté sur l'arbre du moteur délivre deux signaux carrés, A et B, décalés d'un quart de période (quadrature) : le nombre de fronts donne l'angle parcouru, leur ordre le sens (figure 2). En comptant chaque front de A et de B (décodage $\times 4$), un codeur de N impulsions par tour placé avant un réducteur de rapport G fournit $\text{CPR} = 4NG$ fronts par tour de sortie : avec $N = 11$ et $G = 30$, $\text{CPR} = 1\,320$, soit $0,27^\circ$ par front.

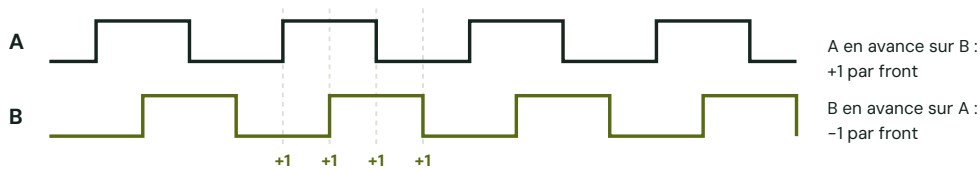


Figure 2 — Voies A et B d'un codeur en quadrature : quatre fronts par impulsion, comptés +1 ou -1 selon le sens.

3.2 Compter les fronts par interruptions

À plusieurs milliers de fronts par seconde, une lecture dans `loop()` en manquerait. Une interruption sur chaque voie exécute donc une routine très courte, qui compare l'état (A, B) précédent au nouveau grâce à une table de 16 cases : +1, -1, ou 0 si les deux voies ont changé à la fois (front manqué ou parasite). Sur l'Uno, seules les broches 2 et 3 acceptent `attachInterrupt()`. Câblage : PWMA sur D9, AIN1 sur D7, AIN2 sur D8, STBY et VCC au 5 V, VM à l'alimentation du moteur, AO1 et AO2 au moteur ; codeur en 5 V, voies A et B sur D2 et D3 ; masses reliées.

commande_pid.ino (1/2) · Arduino Uno et TB6612FNG : codeur et pont en H

```
const byte PIN_A = 2, PIN_B = 3, PIN_PWM = 9, PIN_IN1 = 7, PIN_IN2 = 8;
const float CPR = 4 * 11 * 30.0;           // fronts par tour de sortie : 4 x N x G
const unsigned long TE_US = 10000;         // période d'échantillonnage : 10 ms

volatile long compteur = 0;                 // modifiés par l'interruption
volatile byte etatPrec;

void isrCodeur() {                         // appelée à chaque front de A ou de B
    static const int8_t TABLE[16] = {0, -1, 1, 0, 1, 0, 0, -1, -1, 0, 0, 1, 0, 1, -1, 0};
    byte etat = (digitalRead(PIN_A) << 1) | digitalRead(PIN_B);
    compteur += TABLE[(etatPrec << 2) | etat]; // +1, -1 ou 0 (parasite)
    etatPrec = etat;
}

long lireCompteur() {                       // copie protégée : L'Uno lit un Long de 4 octets en plusieurs fois
    noInterrupts(); long n = compteur; interrupts();
    return n;
}

void commanderMoteur(float u) {              // u en % de la tension, de -100 à +100
    u = constrain(u, -100.0, 100.0);
    digitalWrite(PIN_IN1, u >= 0 ? HIGH : LOW); // sens de rotation
    digitalWrite(PIN_IN2, u >= 0 ? LOW : HIGH);
    analogWrite(PIN_PWM, (int)(fabs(u) * 255 / 100)); // rapport cyclique 0 à 255
}
```

3.3 De la position à la vitesse

Toutes les T_e secondes, l'écart Δn entre deux lectures du compteur donne la vitesse de l'arbre de sortie :

$$\omega_{\text{tr/min}} = \frac{\Delta n \times 60}{\text{CPR} \times T_e}$$

Un front de plus ou de moins vaut $60 / (\text{CPR} \times T_e)$, soit 4,5 tr/min pour $\text{CPR} = 1\,320$ et $T_e = 10$ ms : à 50 tr/min, une dizaine de fronts par période seulement, et la mesure « saute » de $\pm 9\%$. Allonger T_e affine la mesure mais retarde la correction ; à basse vitesse, on peut aussi chronométrer l'intervalle entre deux fronts avec `micros()`.

4. Boucle fermée et PID discret

En **boucle ouverte**, la vitesse obtenue pour un rapport cyclique donné varie avec la charge, la batterie ou la température. En **boucle fermée**, un correcteur compare en permanence la mesure à la consigne et ajuste la commande (figure 3) : il rejette ces perturbations mais, trop énergique, rend la boucle oscillante, voire instable.

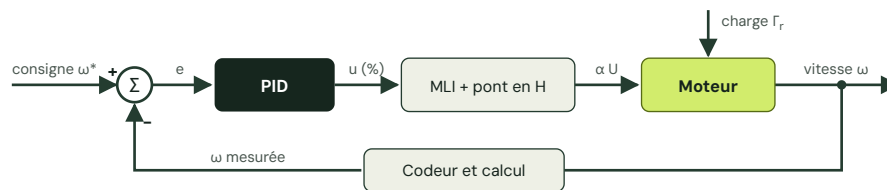


Figure 3 — Boucle de vitesse : le PID agit sur l'écart e entre consigne et mesure, y compris quand la charge change.

4.1 Trois termes complémentaires

$$u(t) = K_p e(t) + K_i \int_0^t e(s) ds + K_d \frac{de}{dt}$$

Le terme **proportionnel** réagit à l'erreur présente ; l'**intégrale** cumule l'erreur passée et annule l'erreur statique ; la **dérivée** réagit à la tendance et amortit les oscillations, mais amplifie le bruit. Pour une vitesse, un PI ($K_d = 0$) suffit le plus souvent ; la dérivée sert surtout en asservissement de position, où le même PID agit sur l'angle (le compteur) au lieu de la vitesse, comme dans un servomoteur de modélisme, qui mesure l'angle par un potentiomètre.

4.2 Discrétisation et dérivée filtrée

Le calcul s'exécute à période fixe T_e , d'un dixième à un cinquième de τ environ. L'intégrale devient une somme : $I_k = I_{k-1} + K_i T_e e_k$. On dérive la *mesure* y plutôt que l'erreur, pour éviter un à-coup de commande à chaque changement de consigne, et on la filtre par un passe-bas de constante de temps T_f :

$$D_k = a D_{k-1} - (1 - a) K_d \frac{y_k - y_{k-1}}{T_e} \quad \text{avec} \quad a = \frac{T_f}{T_f + T_e}$$

4.3 Saturation et anti-emballement de l'intégrale

La commande est bornée à $\pm 100\%$. Si la consigne est inatteignable, l'intégrale grossit sans fin (emballement, *windup*) ; à la baisse de consigne, elle doit d'abord « se vider » et le moteur reste trop longtemps à fond. Remède le plus simple, l'**intégration conditionnelle** : on n'intègre pas une erreur qui pousserait la commande plus loin dans la saturation.

commande_pid.ino (2/2) · PID, mesure de vitesse et boucle à période fixe

```
float Kp = 0.5, Ki = 10.0, Kd = 0.0;           // gains de départ, u en %, vitesse en tr/min
const float U_MAX = 100.0, TF = 0.02;         // saturation (%) ; filtre de la dérivée (s)
float integrale = 0, derivee = 0, mesurePrec = 0;

float pid(float consigne, float mesure, float te) {
    float e = consigne - mesure;
    float a = TF / (TF + te);                  // dérivée de la mesure, filtrée
    derivee = a * derivee - (1 - a) * Kd * (mesure - mesurePrec) / te;
    mesurePrec = mesure;
    float iNouv = integrale + Ki * e * te;
    float u = Kp * e + iNouv + derivee;
    if (u > U_MAX) { u = U_MAX; if (e > 0) iNouv = integrale; } // anti-emballement :
    else if (u < -U_MAX) { u = -U_MAX; if (e < 0) iNouv = integrale; } // pas d'intégration
    integrale = iNouv;                          // vers la saturation
    return u;
}

// Boucle à période fixe : mesure de la vitesse, calcul du PID, commande du
// moteur, puis envoi de consigne, vitesse et commande au traceur série
unsigned long tPrec;
long nPrec = 0;
```

```

void setup() {
  pinMode(PIN_A, INPUT_PULLUP);  pinMode(PIN_B, INPUT_PULLUP);
  pinMode(PIN_IN1, OUTPUT);      pinMode(PIN_IN2, OUTPUT);
  etatPrec = (digitalRead(PIN_A) << 1) | digitalRead(PIN_B);
  attachInterrupt(digitalPinToInterrupt(PIN_A), isrCodeur, CHANGE);
  attachInterrupt(digitalPinToInterrupt(PIN_B), isrCodeur, CHANGE);
  Serial.begin(115200);
  tPrec = micros();
}

void loop() {
  if (micros() - tPrec < TE_US) return;    // attend la fin de la période
  tPrec += TE_US;
  const float te = TE_US * 1e-6;          // en secondes
  long n = lireCompteur();
  float vitesse = (n - nPrec) * 60.0 / (CPR * te);    // tr/min
  nPrec = n;
  float consigne = (millis() / 2000) % 2 ? 150.0 : 50.0; // échelons toutes les 2 s
  float u = pid(consigne, vitesse, te);
  commanderMoteur(u);
  Serial.print("consigne:"); Serial.print(consigne);    // format du traceur série
  Serial.print(" vitesse:"); Serial.print(vitesse);
  Serial.print(" commande:"); Serial.println(u);
}

```

Avant de fermer la boucle, remplacez temporairement `pid(consigne, vitesse, te)` par une commande fixe, par exemple `30`, et vérifiez au traceur série que la vitesse mesurée est *positive*. Sinon, échangez les fils A et B : avec un signe inversé, la boucle pousse le moteur à fond au lieu de le corriger.

Sur ESP32 (cœur Arduino 3.x), la logique est en 3,3 V et les entrées ne tolèrent pas 5 V : VCC du TB6612FNG en 3,3 V, codeur en 3,3 V ou niveaux adaptés. Sur l'ESP32 classique, les GPIO 6 à 11 (mémoire flash), 3 (liaison série USB) et 2 (broche de démarrage) sont à éviter : changez toutes les broches du programme et déclarez `void IRAM_ATTR isrCodeur()`. Pour une MLI à 20 kHz, `ledcAttach(PIN_PWM, 20000, 10)`; dans `setup()`, puis `ledcWrite(PIN_PWM, (int)(fabs(u) * 1023 / 100))`; au lieu d' `analogWrite`.

5. Régler le PID

5.1 Réglage manuel

Partez de $K_i = K_d = 0$ et augmentez K_p jusqu'à une réponse rapide, sans oscillation ni commande qui « vibre ». Ajoutez K_i jusqu'à annuler l'erreur statique sans dépassement excessif, puis K_d seulement si nécessaire, avec son filtre. Changez un seul gain à la fois et notez chaque essai. Tendances, à vérifier sur votre système : plus de K_p accélère et réduit l'erreur statique sans l'annuler, mais accroît dépassement et bruit ; K_i annule l'erreur statique au prix d'un dépassement accru ; un K_d modéré réduit le dépassement mais amplifie fortement le bruit.

5.2 La méthode de Ziegler-Nichols

Publiée en 1942, la méthode du gain critique se pratique en boucle fermée : avec $K_i = K_d = 0$, on augmente K_p jusqu'à des oscillations d'amplitude constante, on relève ce gain critique K_u et la période T_u , puis on applique le tableau, avec $K_i = K_p / T_i$ et $K_d = K_p T_d$.

Réglages de Ziegler-Nichols par la méthode du gain critique

Correcteur	K_p	T_i	T_d
P	$0,5 K_u$	—	—
PI	$0,45 K_u$	$T_u / 1,2$	—
PID	$0,6 K_u$	$T_u / 2$	$T_u / 8$

Ces réglages visent une oscillation réduite au quart à chaque période, donc un net dépassement, à adoucir ensuite. Sur une boucle de vitesse, seuls les retards (échantillonnage, mesure) font osciller : K_u est élevé, T_u vaut quelques T_e et la résolution du codeur brouille l'observation. Essayez à faible consigne, la main sur l'interrupteur.

5.3 À partir d'un modèle : la méthode lambda

Connaissant K et τ , on choisit la constante de temps λ voulue en boucle fermée ; un PI dont le zéro compense le pôle du moteur, $K_p = \tau / (K\lambda)$ et $K_i = 1 / (K\lambda)$, donne alors, sans saturation, une réponse du premier ordre de constante de temps λ , sans dépassement. $\lambda = \tau$ est un choix prudent ; un λ plus petit accélère la boucle jusqu'à saturer la commande et amplifie le bruit. Les gains du programme ($K_p = 0,5$, $K_i = 10$) correspondent à $K = 3$ tr/min par %, $\tau = 50$ ms et $\lambda \approx 33$ ms.

EXERCICE GUIDÉ · IDENTIFIER PUIS ASSERVIR VOTRE MOTEUR

Montage des sections 2 et 3, programme `commande_pid.ino`, traceur série à 115 200 bauds.

- Boucle ouverte.** Remplacez `pid(consigne, vitesse, te)` par `((millis() / 2000) % 2 ? 60.0 : 30.0)`. Relevez l'écart de vitesse $\Delta\omega$ entre les deux paliers et le temps mis pour en parcourir 63 % ; calculez $K = \Delta\omega / 30$ et τ .
- Méthode lambda.** Calculez K_p et K_i pour $\lambda = \tau$, rétablissez l'appel à `pid` et observez les échelons de 50 à 150 tr/min. Recommencez avec $\lambda = \tau/2$ puis $\tau/4$: à partir de quand la commande sature-t-elle ?
- Anti-emballement.** Remplacez `150.0` par une consigne inatteignable, nettement supérieure à 100 K. Mesurez le temps mis par la vitesse pour repasser sous 60 tr/min après chaque baisse de consigne, puis supprimez les deux instructions `iNouv = integrale;` et comparez.
- Ziegler-Nichols (facultatif).** Avec $K_i = 0$ et 50 tr/min de consigne, cherchez K_u et T_u ; comparez au réglage lambda.

Critères de réussite : vitesse positive pour une commande positive ; avec $\lambda = \tau$, 63 % de l'échelon en un temps proche de λ (à 30 % près), dépassement d'un ou deux fronts au plus, vitesse établie centrée sur la consigne à un ou deux fronts près ; avec l'anti-emballement, retour sous 60 tr/min en moins de 0,5 s, nettement plus lent sans.

Indice : avec $K = 3$ tr/min par % et $\tau = 50$ ms, $\lambda = \tau$ donne $K_p \approx 0,33$ et $K_i \approx 6,7$; $\lambda = \tau/2$, 0,67 et 13. À l'échelon de 100 tr/min, la commande bondit d'environ 120 K_p points depuis environ 17 % (100 K_p de terme proportionnel, plus 20 K_p d'intégrale dès la première période, car $K_i T_e = 0,2 K_p$) : elle atteint la saturation dès $K_p \approx 0,7$, soit $\lambda \approx \tau/2$. Une vitesse qui saute d'un front à l'autre traduit la résolution du codeur (section 3.3), pas un défaut de réglage.

6. Ce que l'IA apporte à la commande

Pour un moteur seul, un PI bien réglé suffit souvent : simple, explicable, éprouvé. L'IA devient utile quand le système est mal connu, évolue dans le temps, ou quand les boucles à régler sont nombreuses.

6.1 Identifier le modèle à partir des données

Identifier, c'est estimer les paramètres d'un modèle à partir de mesures : une régression, brique de base de l'apprentissage automatique. Ici, les paramètres du modèle discret $y_{k+1} = a y_k + b u_k$ sont estimés par moindres carrés sur des paliers de commande et des mesures bruitées simulées ; on en déduit K et τ , puis les gains par la méthode lambda.

identification.py

```
import numpy as np

rng = np.random.default_rng(1)
TE, K_VRAI, TAU_VRAI = 0.01, 3.0, 0.05 # « moteur » simulé : gain (tr/min par %), tau (s)
u = np.repeat(rng.choice([30.0, 60.0, 90.0]), 30), 10 # paliers de commande de 0,1 s (%)
a = np.exp(-TE / TAU_VRAI)
```

```
w = np.zeros(u.size)
for k in range(u.size - 1):
    w[k + 1] = a * w[k] + K_VRAI * (1 - a) * u[k]
y = w + rng.normal(0.0, 3.0, w.size) # mesure bruitée (tr/min)

# Modèle  $y[k+1] = a.y[k] + b.u[k]$  : deux paramètres estimés par moindres carrés
X = np.column_stack([y[:-1], u[:-1]])
(a_est, b_est), *_ = np.linalg.lstsq(X, y[1:], rcond=None)
K, tau = b_est / (1 - a_est), -TE / np.log(a_est)
lam = tau # méthode lambda, choix prudent
print(f"K = {K:.2f} tr/min par %, tau = {1000 * tau:.0f} ms")
print(f"PI : Kp = {tau / (K * lam):.2f}, Ki = {1 / (K * lam):.1f}")
```

```
K = 2.99 tr/min par %, tau = 50 ms
PI : Kp = 0.33, Ki = 6.7
```

Sur un vrai moteur, utilisez les mesures de l'étape 1 de l'exercice. Un réseau de neurones peut capturer en plus les non-linéarités (frottement sec, zone morte à faible commande), au prix de plus de données et d'une interprétation plus difficile.

6.2 Régler automatiquement, apprendre par renforcement

Sur le modèle identifié, une optimisation (bayésienne, par exemple) peut chercher les gains qui minimisent l'intégrale de $|e|$, en simulation puis sur le banc avec des gains bornés ; l'autorégulation par relais d'Åström et Hägglund en est l'équivalent classique. En apprentissage par renforcement, un agent apprend par essais et erreurs, dans un simulateur (jumeau numérique), une politique qui maximise une récompense comme $-|e| - c u^2$: utile pour des systèmes non linéaires à plusieurs entrées, comme un robot à pattes, superflu pour une boucle de vitesse, où le PID reste plus sûr et plus facile à justifier. Le cours WILLSIM « Apprentissage par renforcement et jumeau numérique » approfondit cette voie.

À RETENIR

- Moteur à courant continu : premier ordre $K/(1 + \tau p)$, courant de blocage U/R .
- MLI : tension moyenne αU ; pont en H : sens, sans jamais fermer deux interrupteurs d'un même bras.
- Codeur : décodage $\times 4$ par interruptions, vitesse = $\Delta n \times 60 / (CPR \times T_e)$, signe vérifié avant de fermer la boucle.
- PID à période fixe, commande bornée, anti-emballement, dérivée de la mesure filtrée ; souvent, un PI suffit.
- Mesurer, identifier, calculer, vérifier. L'IA aide à identifier et à régler ; elle ne remplace ni la compréhension du système ni l'arrêt d'urgence.

Pour aller plus loin

- [Arduino, référence de attachInterrupt\(\)](https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/) — <https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/>
- [Arduino, « Secrets of Arduino PWM » \(minuteries et fréquences de MLI\)](https://docs.arduino.cc/tutorials/generic/secrets-of-arduino-pwm/) — <https://docs.arduino.cc/tutorials/generic/secrets-of-arduino-pwm/>
- [K. J. Åström et R. M. Murray, Feedback Systems \(accès libre\)](https://fbswiki.org/wiki/index.php/Main_Page) — https://fbswiki.org/wiki/index.php/Main_Page
- [Page du cours sur willsim.fr](https://willsim.fr/cours/commande-moteurs-pid.html) — <https://willsim.fr/cours/commande-moteurs-pid.html>

Fiche rédigée par l'équipe pédagogique WILLSIM, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécatronique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 (creativecommons.org/licenses/by-nc-sa/4.0/deed.fr) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à contact@willsim.fr · willsim.fr